



**KCC INSTITUTE OF TECHNOLOGY AND
MANAGEMENT**



Technical Training And Placement
2nd Yr Week2 Module

Submitted By:

ICHCHA SHAH

ROLL NO : 2504921530157

Week-2 Module

1. Find student roll number (Linear search)

Problem statement:

Your class teacher has a list of student roll no. stored in the order they registered since the list is unsorted, you need to find whether a particular student roll no. exist.

Write a program to search for the given roll number using linear search.

Code → #include <stdio.h>

```
int main () {
```

// Array of student roll numbers.

```
int arr[5] = {101, 105, 113, 110, 130};
```

// size of the array

```
int n=5;
```

// Roll number to search

```
int searchvalue = 110;
```

// Flag to indicate if the value is found

```
int found = 0;
```

// Traverse the array one by one

```
for (int i=0; i<n; i++) {
```

// Check if current element matches the search value

```
if (arr[i] == searchvalue) {
```

// Mark element as found and print the index

```
found = 1;
```

```
printf("Element found at index %d\n", i);
```

// stop searching after finding the element
break;

}

}

// if element is not found, print a message

if (!found) {

printf("Element not found in array\n");

}

return 0;

}

```
File Edit Selection View Go Run Terminal Help C:\programs
C linearsearch.c x C binarysearch.c
C linearsearch.c > [main]
1 #include <stdio.h>
2 int main() {
3     // Array of student roll numbers
4     int arr[5] = {101, 105, 113, 110, 130};
5
6     // Size of the array
7     int n = 5;
8
9     // Roll number to search
10    int searchValue = 110;
11
12    // Flag to indicate if the value is found
13    int found = 0;
14
15    // Traverse the array one by one
16    for (int i = 0; i < n; i++) {
17
18        // Check if current element matches the search value
19        if (arr[i] == searchValue) {
20
21            // Mark element as found and print the index
22            found = 1;
23            printf("Element found at index %d\n", i);
24
25            // Stop searching after finding the element
26            break;
27        }
28    }
29    // if element is not found, print a message
30    if (!found) {
31        printf("Element not found in array\n");
32    }
33    return 0;
34 }
```

PS C:\Users\ichcha\Desktop\c_programs> C:\Users\ichcha\Desktop\c_programs\linearsearch.exe
Element found at index 3
PS C:\Users\ichcha\Desktop\c_programs>

2. Search Product ID (Binary Search)

Problem Statement:

An online store keeps product IDs sorted in ascending order. A customer wants to know whether a particular product exists in inventory.

Write a program to search the product using Binary search.

Code

```
#include <stdio.h>
int main() {
    int arr[6] = {10, 20, 30, 40, 50, 60};
    int low = 0, high = 6-1, mid, searchvalue = 40;
    int found = 0; // flag to indicate if the value
                  // is found
    while (low <= high) {
        mid = low + (high - low) / 2;
        if (arr[mid] == searchvalue) {
            found = 1;
        }
    }
}
```

```
print("Product Available\n");
```

```
break;
```

```
}  
else if (arr[mid] < searchValue) {  
    low = mid + 1;
```

```
}  
else {  
    high = mid - 1;
```

```
}
```

```
}
```

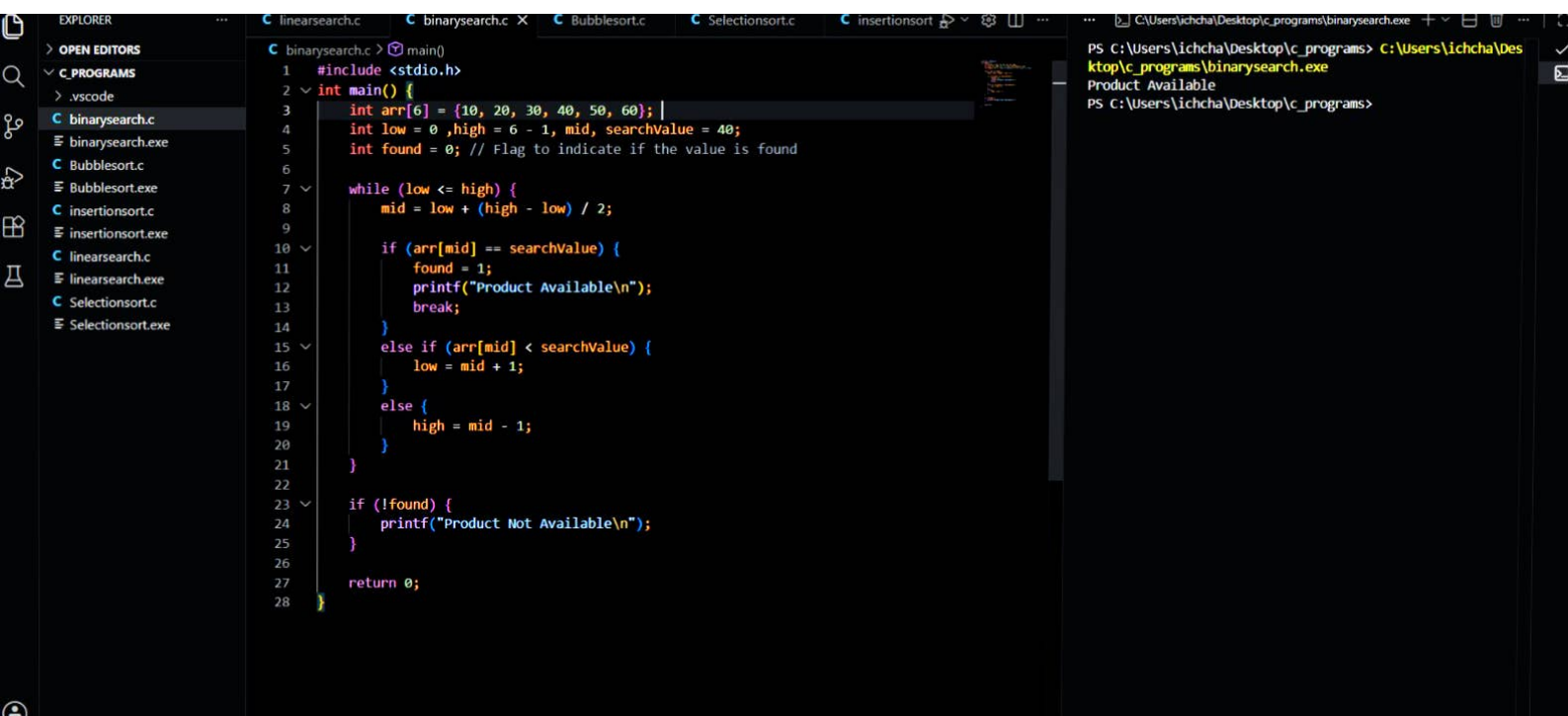
```
if (!found) {
```

```
    print("Product Not Available\n");
```

```
}
```

```
return 0;
```

```
}
```



3. Arrange Exam Scores (Bubble sort)

Problem statement:

A teacher wants to arrange students' exam scores in ascending order to prepare the results sheet.

Write a program using Bubble sort

code #include <stdio.h>

int main () {

int arr[5] = {89, 45, 78, 12, 99};

int n = 5;

for (int i = 0; i < n - 1; i++) {

for (int j = 0; j < n - i - 1; j++) {

if (arr[j] > arr[j + 1]) {

// swap arr[j] and arr[j + 1]

int temp = arr[j];

arr[j] = arr[j + 1];

arr[j + 1] = temp;

}

printf("Sorted array: \n");

for (int i = 0; i < n; i++) {

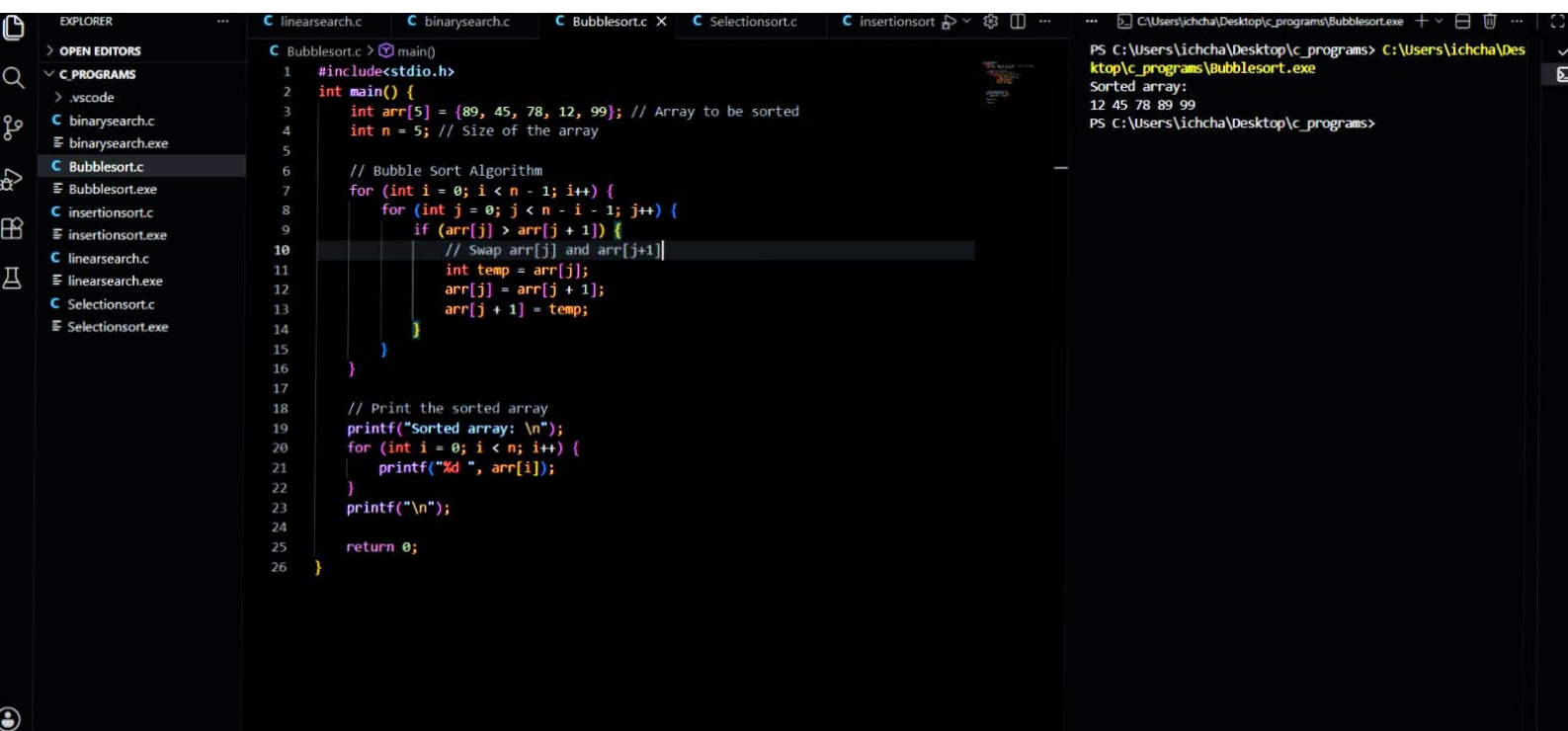
printf("%d", arr[i]);

}

printf("\n");

return 0;

}



4. Arrange Library Book Numbers (Insertion sort)

Problem statement:-

A Librarian receives book serial numbers one by one and wants to place each new book in the correct position.

Write a program using Insertion Sort to arrange book numbers.

Code ~~#include <stdio.h>~~

```
int main() {
```

```
    int arr[5] = {55, 23, 87, 11, 34};
```

```
    int n=5;
```

```
    for (int i=1; i<n; i++) {
```

```
        int key = arr[i];
```

```
        int j = i-1;
```

// More elements of arr[0..i-1], that are
// greater than key, to one position ahead
// to their current position

```
        while (j >= 0 && arr[j] > key) {
```

```
            arr[j+1] = arr[j];
```

```
            j = j-1;
```

```
        }
```

```
        arr[j+1] = key;
```

```
    }  
    printf("sorted book numbers:\n");
```

```
    for (int i = 0; i < n; i++)
```

```
    }
```

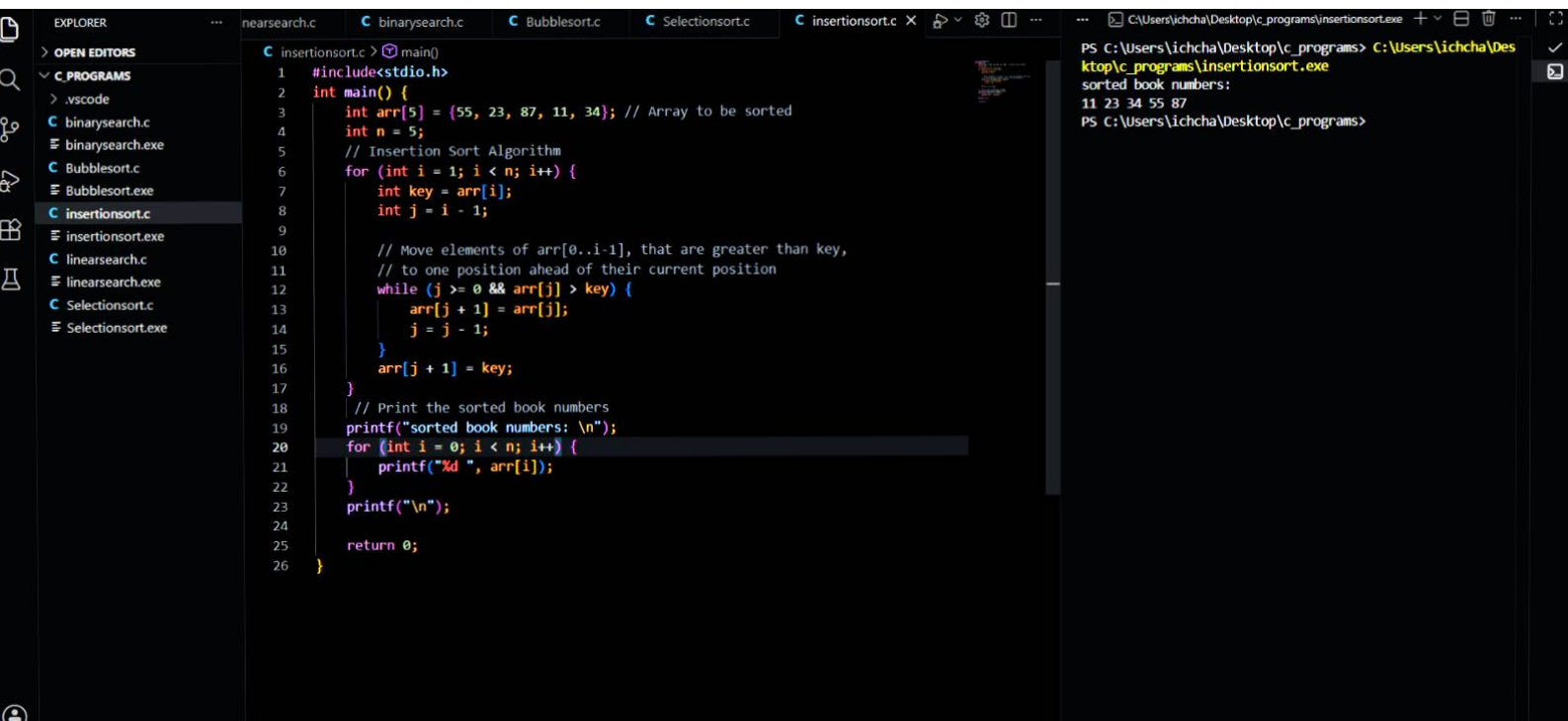
```
printf("%d", arr[i]);
```

```
}
```

```
printf("\n");
```

```
return 0;
```

```
}
```



5. Rank Players by score (selection sort)

Problem statement ?

A gaming club wants to rank players based on scores from lowest to highest.

Write a program using selection sort.

Code #include <stdio.h>

int main() {

int arr[5] = {70, 20, 90, 10, 40};

int n = 5;

int i, j, minIndex, temp;

for (i = 0; i < n - 1; i++) {

minIndex = i;

for (j = i + 1; j < n; j++) {

if (arr[j] < arr[minIndex]) {

minIndex = j;

}

// swap the found min element with the 1st

temp = arr[minIndex];

arr[minIndex] = arr[i];

arr[i] = temp;

}

```
printf("sorted scores:\n");
```

```
for (i=0; i<n; i++) {
```

```
    printf("%d", arr[i]);
```

```
}
```

```
printf("\n");
```

```
return 0;
```

```
}
```

